

Practical Demo: Callable-Bond Valuation, OAS, and Effective Risk

Table of contents

Learning Objectives	1
1 Market Inputs Versus Illustrative Inputs	1
2 Build a Recombining Short-Rate Tree	2
3 Price the Option-Free and Callable Bonds	4
4 Illustrate and Solve for OAS	6
5 Effective Duration and Effective Convexity	8
6 Validation Checks	10
7 Financial Interpretation	10

Learning Objectives

After completing this demo you should be able to:

- value option-free and callable bonds with backward induction;
- apply the issuer's call decision at each eligible node;
- isolate the embedded call option's price cost;
- solve for the option-adjusted spread (OAS) that matches a market price; and
- calculate effective duration and effective convexity by repricing the bond.

1 Market Inputs Versus Illustrative Inputs

This is a transparent teaching workflow, not a live security valuation. A production model would calibrate an arbitrage-free tree to a current benchmark curve and an option-volatility surface.

Input	Value	Status
Face value	100	Illustrative bond term
Annual coupon rate	5.25%	Illustrative bond term
Final maturity	7 years	Illustrative bond term
Call schedule	Years 3–6	Illustrative bond term
Call prices	101.50, 101.00, 100.50, 100.00	Illustrative bond term
Median one-year short-rate path	4.20% to 3.70%	Market-like, not live
Annual short-rate volatility	18%	Illustrative model input
Risk-neutral up probability	50%	Simplifying assumption
Observed callable-bond price	Generated below	Illustrative market price

The rate tree is recombining and lognormal. It captures path-dependent call decisions, but it is not presented as a fully calibrated term-structure model. Rates cannot be negative, the time step is one year, the OAS is a constant spread, and transaction costs, taxes, accrued interest, default, recovery, and notice periods are ignored.

Workflow stage	In this case
Supplied in the teaching case	Callable price and contractual cash flows
Assumed	Benchmark rate tree, volatility, and exercise convention
Solved	OAS that reprices the observed bond price
Diagnosed	Repricing error and call exercise by node
Used for	Effective duration, convexity, and rate-stress decisions

2 Build a Recombining Short-Rate Tree

At time t , node j has short rate

$$r_{t,j} = m_t \exp[(2j - t)\sigma],$$

where m_t is the median-rate path and σ controls annual volatility. Moving up increases j by one; moving down leaves j unchanged. Because the same number of up moves leads to the same node, the tree recombines.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

face = 100.0
coupon_rate = 0.0525
coupon = face * coupon_rate
maturity = 7
median_rates = np.array([0.0420, 0.0410, 0.0400, 0.0390, 0.0380, 0.0375, 0.0370])
rate_volatility = 0.18
up_probability = 0.50

call_schedule = {3: 101.50, 4: 101.00, 5: 100.50, 6: 100.00}

def build_rate_tree(medians, volatility, parallel_shift=0.0):
    """Return a recombining lognormal short-rate tree."""
    tree = []
    for t, median in enumerate(medians):
        nodes = np.array([
            median * np.exp((2 * j - t) * volatility) + parallel_shift
            for j in range(t + 1)
        ])
        if np.any(nodes <= -1.0):
            raise ValueError("A shocked node rate is below -100%.")
        tree.append(nodes)
    return tree

rate_tree = build_rate_tree(median_rates, rate_volatility)
rate_table = pd.DataFrame({
    f"Year {t}": pd.Series(100 * nodes)
    for t, nodes in enumerate(rate_tree)
}).round(3)
rate_table.index.name = "Node (number of up moves)"
rate_table
```

	Year 0	Year 1	Year 2	Year 3	Year 4	Year 5	Year 6
Node (number of up moves)							
0	4.2	3.425	2.791	2.273	1.850	1.525	1.257
1	NaN	4.909	4.000	3.258	2.651	2.185	1.801
2	NaN	NaN	5.733	4.669	3.800	3.132	2.581
3	NaN	NaN	NaN	6.692	5.447	4.490	3.700
4	NaN	NaN	NaN	NaN	7.807	6.435	5.303
5	NaN	NaN	NaN	NaN	NaN	9.224	7.601
6	NaN	NaN	NaN	NaN	NaN	NaN	10.895

The entries are annual short rates in percent. The low-rate nodes are economically important because they are where refinancing and call exercise become most attractive.

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
for t, nodes in enumerate(rate_tree):
    y = np.arange(t + 1) - t / 2
    ax.scatter(np.repeat(t, t + 1), y, color="#4C78A8", s=45, zorder=3)
    for j, (yj, rate) in enumerate(zip(y, nodes)):
        ax.text(t, yj + 0.14, f"{100*rate:.2f}%", ha="center", fontsize=8)
        if t < maturity - 1:
            ax.plot([t, t + 1], [yj, j - (t + 1) / 2], color="#BBBBBB", linewidth=0.8)
            ax.plot([t, t + 1], [yj, j + 1 - (t + 1) / 2], color="#BBBBBB", linewidth=0.8)
ax.set(xlabel="Year", title="Short rates by state")
ax.set_yticks([])
ax.grid(axis="x", alpha=0.2)
plt.tight_layout()
plt.show()
```

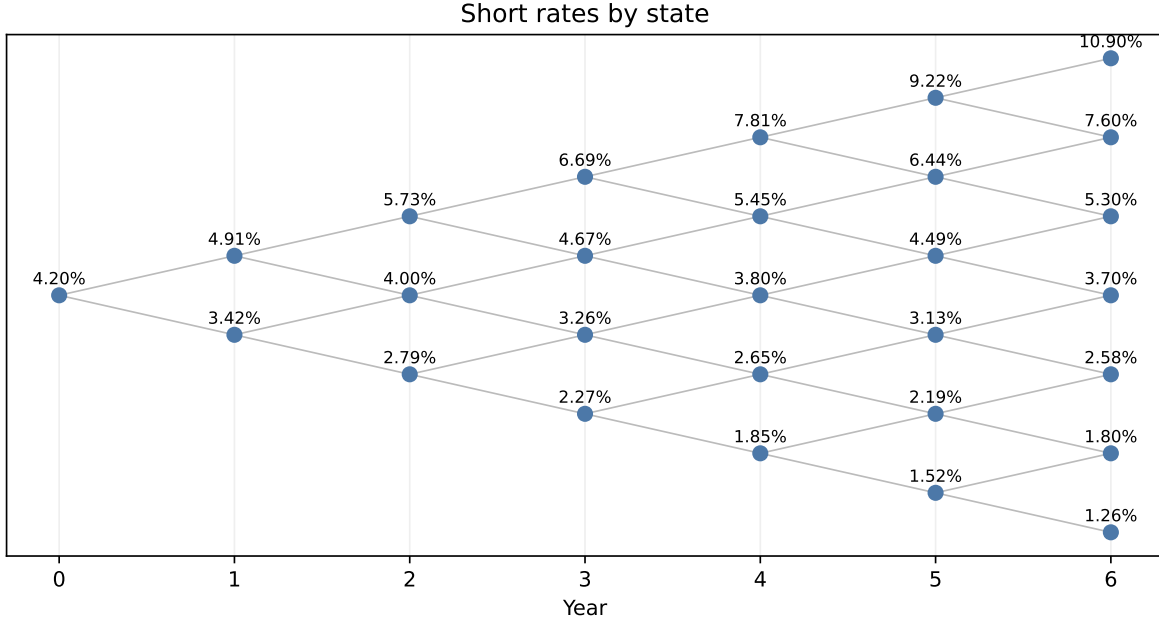


Figure 1: Illustrative recombining short-rate tree

3 Price the Option-Free and Callable Bonds

Values are measured immediately after any coupon at a node. The one-period holding value is

$$V_{t,j}^{\text{hold}} = \frac{C + pV_{t+1,j+1} + (1-p)V_{t+1,j}}{1 + r_{t,j} + s}.$$

For the option-free bond, this holding value is retained. At an eligible call date, the callable bond applies the issuer's minimum rule:

$$V_{t,j}^{\text{callable}} = \min \{ V_{t,j}^{\text{hold}}, \text{Call Price}_t \}.$$

```
def price_bond(rate_tree, spread=0.0, callable_bond=False, return_tree=False):
    """Price the bond by backward induction; node values are ex-coupon."""
    values = np.full(maturity + 1, face, dtype=float)
    value_tree = [None] * (maturity + 1)
    value_tree[maturity] = values.copy()
```

```

for t in range(maturity - 1, -1, -1):
    expected_next = (
        (1 - up_probability) * values[:-1]
        + up_probability * values[1:]
    )
    hold_value = (coupon + expected_next) / (1 + rate_tree[t] + spread)
    if callable_bond and t in call_schedule:
        values = np.minimum(hold_value, call_schedule[t])
    else:
        values = hold_value
    value_tree[t] = values.copy()

if return_tree:
    return float(values[0]), value_tree
return float(values[0])

benchmark_straight_price = price_bond(rate_tree, spread=0.0)
benchmark_callable_price = price_bond(rate_tree, spread=0.0, callable_bond=True)
benchmark_option_cost = benchmark_straight_price - benchmark_callable_price

pd.Series({
    "Option-free value": benchmark_straight_price,
    "Callable value": benchmark_callable_price,
    "Embedded call value": benchmark_option_cost,
}).round(4)

```

```

Option-free value      107.0091
Callable value         103.3419
Embedded call value     3.6672
dtype: float64

```

The price decomposition is

$$V_{\text{callable}} = V_{\text{option-free}} - V_{\text{call option}}.$$

The option cost is positive because the issuer owns the call. It is largest when low-rate paths make refinancing valuable.

4 Illustrate and Solve for OAS

To make the exercise reproducible, suppose the callable bond trades at the model value generated by an 85-basis-point spread. We then hide that spread and recover it from the price. In practice, the price would come from the market.

```
illustrative_true_oas = 0.0085
market_price = price_bond(
    rate_tree,
    spread=illustrative_true_oas,
    callable_bond=True,
)

def solve_spread(target_price, callable_bond, lower=-0.02, upper=0.10, tolerance=1e-12):
    """Solve a constant spread with robust bisection."""
    price_lower = price_bond(rate_tree, lower, callable_bond)
    price_upper = price_bond(rate_tree, upper, callable_bond)
    if not (price_lower >= target_price >= price_upper):
        raise ValueError("Target price is not bracketed by the spread interval.")

    for _ in range(200):
        midpoint = 0.5 * (lower + upper)
        midpoint_price = price_bond(rate_tree, midpoint, callable_bond)
        if abs(midpoint_price - target_price) < tolerance:
            return midpoint
        if midpoint_price > target_price:
            lower = midpoint
        else:
            upper = midpoint
    return 0.5 * (lower + upper)

solved_oas = solve_spread(market_price, callable_bond=True)
static_spread = solve_spread(market_price, callable_bond=False)
option_cost_spread = static_spread - solved_oas
reprice = price_bond(rate_tree, solved_oas, callable_bond=True)

pd.Series({
    "Illustrative market price": market_price,
    "Solved OAS (bp)": solved_oas * 10_000,
    "Static spread on promised cash flows (bp)": static_spread * 10_000,
    "Option cost in spread terms (bp)": option_cost_spread * 10_000,
    "Repriced callable value": reprice,
```

```
}).round(6)
```

Illustrative market price	100.0700
Solved OAS (bp)	85.0000
Static spread on promised cash flows (bp)	115.8285
Option cost in spread terms (bp)	30.8285
Repriced callable value	100.0700
dtype: float64	

OAS is the spread left after the tree has modeled the issuer's exercise decision. The static spread is larger because it discounts promised maturity cash flows and does not recognize that favorable low-rate paths can be taken away through a call.

The option cost can also be expressed in price terms at the solved OAS:

```
straight_at_oas = price_bond(rate_tree, solved_oas, callable_bond=False)
callable_at_oas = price_bond(rate_tree, solved_oas, callable_bond=True)
option_cost_at_oas = straight_at_oas - callable_at_oas

pd.Series({
    "Option-free value at OAS": straight_at_oas,
    "Callable value at OAS": callable_at_oas,
    "Call option cost in price points": option_cost_at_oas,
}).round(4)
```

Option-free value at OAS	101.8603
Callable value at OAS	100.0700
Call option cost in price points	1.7903
dtype: float64	

5 Effective Duration and Effective Convexity

Modified duration is not sufficient because the call decision changes across rate paths. Hold OAS fixed, shift the entire benchmark tree by $\pm\Delta y$, and fully reprice:

$$D_{\text{eff}} = \frac{P_- - P_+}{2P_0\Delta y},$$
$$C_{\text{eff}} = \frac{P_- + P_+ - 2P_0}{P_0(\Delta y)^2}.$$


```

curve_shock = 0.0025 # 25 basis points

def effective_risk(callable_bond):
    base = price_bond(rate_tree, solved_oas, callable_bond)
    down_tree = build_rate_tree(median_rates, rate_volatility, -curve_shock)
    up_tree = build_rate_tree(median_rates, rate_volatility, curve_shock)
    price_down = price_bond(down_tree, solved_oas, callable_bond)
    price_up = price_bond(up_tree, solved_oas, callable_bond)
    duration = (price_down - price_up) / (2 * base * curve_shock)
    convexity = (price_down + price_up - 2 * base) / (base * curve_shock**2)
    return base, price_down, price_up, duration, convexity

straight_risk = effective_risk(False)
callable_risk = effective_risk(True)

risk_table = pd.DataFrame(
    [straight_risk, callable_risk],
    index=["Option-free bond", "Callable bond"],
    columns=["P0", "P(-25 bp)", "P(+25 bp)", "Effective duration", "Effective convexity"],
)
risk_table.round(4)

```

	P0	P(-25 bp)	P(+25 bp)	Effective duration	Effective convexity
Option-free bond	101.8603	103.3417	100.4054	5.7655	41.7602
Callable bond	100.0700	101.0662	99.0611	4.0074	-20.2338

The callable bond should generally have shorter effective duration because its upside is capped when rates fall. Its effective convexity can also be much lower, or negative, near the region where call exercise changes rapidly.

```

shocks_bp = np.arange(-150, 151, 10)
straight_prices = []
callable_prices = []
for shock_bp in shocks_bp:
    shocked_tree = build_rate_tree(median_rates, rate_volatility, shock_bp / 10_000)
    straight_prices.append(price_bond(shocked_tree, solved_oas, False))
    callable_prices.append(price_bond(shocked_tree, solved_oas, True))

fig, ax = plt.subplots(figsize=(7.6, 4.2))
ax.plot(shocks_bp, straight_prices, linewidth=2.1, label="Option-free bond")

```

```

ax.plot(shocks_bp, callable_prices, linewidth=2.1, label="Callable bond")
ax.axvline(0, color="gray", linestyle="--", linewidth=1)
ax.set(xlabel="Parallel benchmark-curve shock (bp)", ylabel="Model price",
       title="Callable and option-free repricing at constant OAS")
ax.grid(alpha=0.3)
ax.legend()
plt.tight_layout()
plt.show()

```

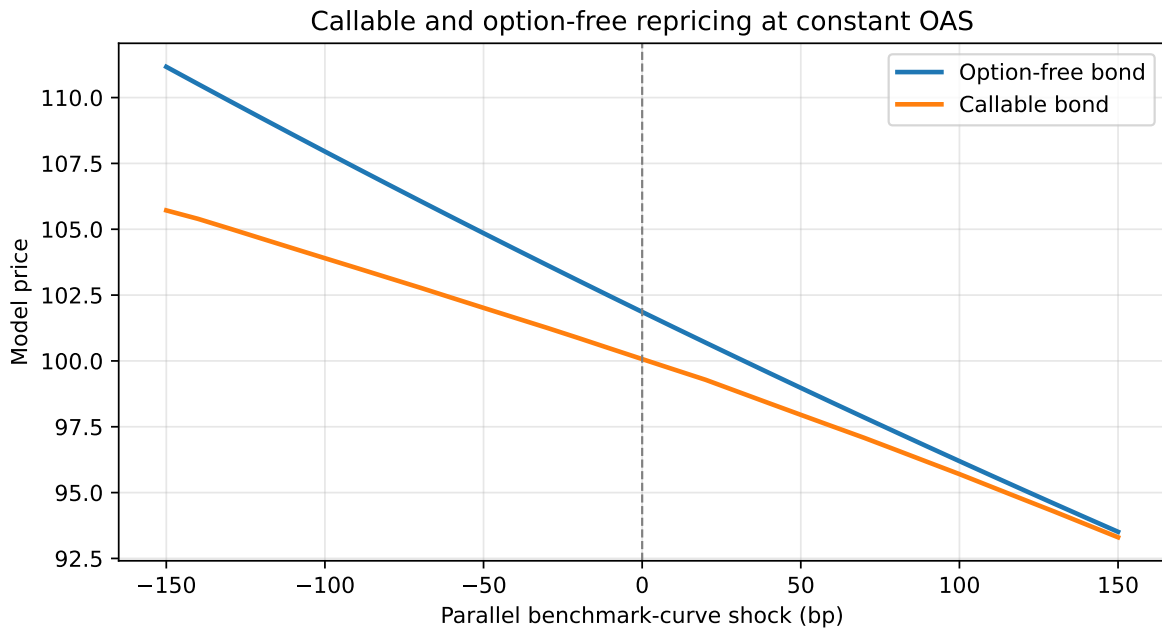


Figure 2: Full repricing reveals the callable bond's capped upside

6 Validation Checks

```

assert abs(solved_oas - illustrative_true_oas) < 1e-10
assert abs(reprice - market_price) < 1e-10
assert benchmark_callable_price <= benchmark_straight_price
assert callable_at_oas <= straight_at_oas
assert option_cost_at_oas >= 0
assert callable_risk[1] > callable_risk[0] > callable_risk[2]

```

```

checks = pd.Series({
    "OAS recovered within 0.0001 bp": abs(solved_oas - illustrative_true_oas) < 1e-8,
    "Model reprices market value": abs(reprice - market_price) < 1e-8,
    "Callable value does not exceed straight value": callable_at_oas <= straight_at_oas,
    "Price falls when the curve rises": callable_risk[2] < callable_risk[0],
    "Price rises when the curve falls": callable_risk[1] > callable_risk[0],
})
checks

```

```

OAS recovered within 0.0001 bp          True
Model reprices market value             True
Callable value does not exceed straight value True
Price falls when the curve rises        True
Price rises when the curve falls        True
dtype: bool

```

7 Financial Interpretation

This workflow separates three ideas that a yield-to-maturity comparison mixes together:

1. The benchmark short-rate tree values state-contingent cash flows.
2. The minimum rule transfers low-rate upside from the investor to the issuer.
3. OAS measures the constant spread that remains after the call option has been modeled.

The outputs are model-dependent. A different benchmark curve, volatility, call behavior, or tree calibration can materially change the option value, OAS, effective duration, and effective convexity.